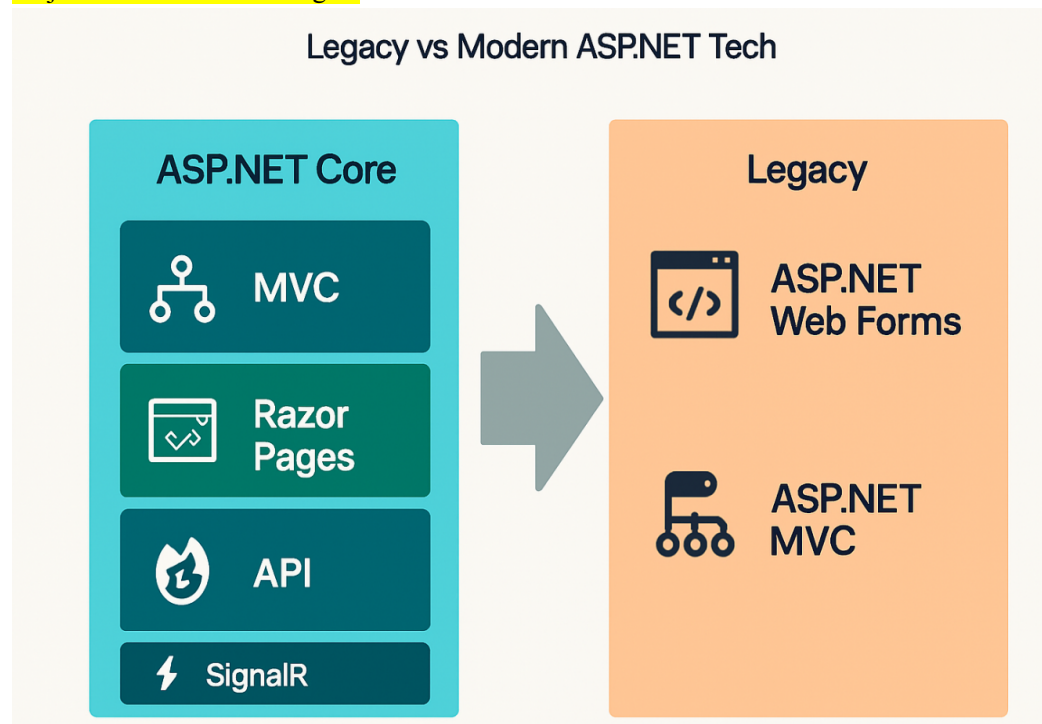


ASP.NET Technologies

ASP.NET is a set of frameworks within the .NET ecosystem designed for building web applications, services, and APIs. Over time, ASP.NET has evolved to support different development paradigms and requirements.

Major ASP.NET Technologies



1. ASP.NET Web Forms

- **Description:** A UI-focused, event-driven framework enabling rapid development of web forms and interactive web applications.
- **Features:**
 - Drag-and-drop controls
 - ViewState for managing state across requests
 - Server-side controls and events
- 1. **Server Controls**- ASP.NET Web server controls are similar to familiar HTML elements, such as buttons and text boxes. Other controls are calendar controls, and controls that you can use to connect to data sources and display data.
- 2. **Master Pages**- ASP.NET master pages allow you to create a consistent layout for the pages in your application. A single master page defines the look and feel and standard behavior for all of the pages (or a group of pages) in your application. You can then create individual content pages along with the master page to render the web page.
- 3. **Working with Data**- ASP.NET provides many options for storing, retrieving, and displaying data in web page UI elements such as tables and text boxes and drop-down lists
- 4. **Client Script and Client Frameworks**- You can write client-script functionality in ASP.NET Web Form pages to provide responsive user interface to users. You can also use client script to make asynchronous calls to the Web server while a page is running in the browser.

5. **Routing**- URL routing allows you to configure an application to accept request URL. A request URL is simply the URL a user enters into their browser to find a page on your web site. You use routing to define URLs that are semantically meaningful to users and that can help with search-engine optimization (SEO).
- **Common Use:** Enterprise internal apps, business portals

2. ASP.NET MVC

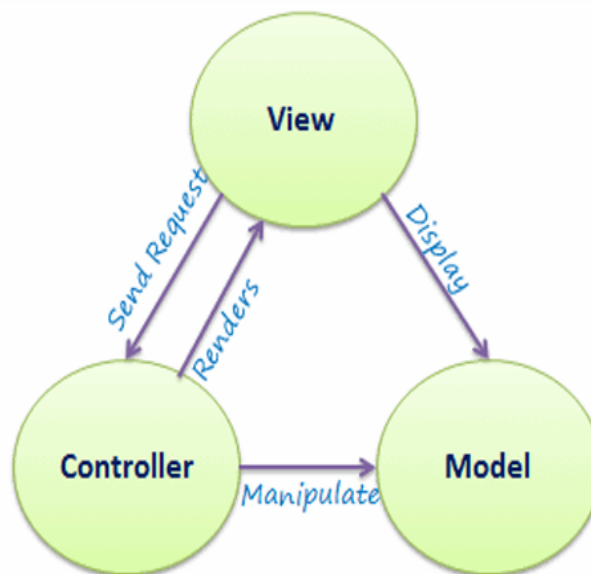
- **Description:** Follows the Model-View-Controller (MVC) pattern, separating application logic, user interface, and input control.

ASP.NET MVC Architecture

MVC stands for Model, View, and Controller. MVC separates an application into three components- Model, View, and Controller.

1. **Model**: represents the shape of the data. A class in C# is used to describe a model. Model objects store data retrieved from the database. Model represents the data.
 2. **View**: View in MVC is a user interface. View display model data to the user and also enables them to modify them. View in ASP.NET MVC is HTML, CSS, and some special syntax (Razor syntax) that makes it easy to communicate with the model and the controller.
 3. **Controller**: handles the user request. Typically, the user uses the view and raises an HTTP request. Controller processes request and returns the appropriate view as a response. Controller is the request handler.
- **Features:**
 - Clean separation of concerns
 - Enhanced unit testing support
 - HTML and JavaScript integration

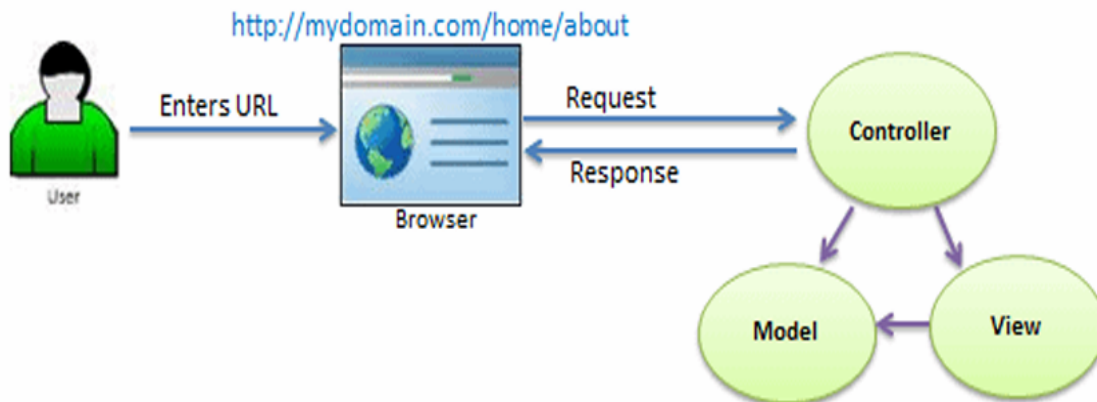
ASP.NET MVC Architecture



- **Common Use:** Websites and web apps needing flexibility and maintainability

Request Flow in MVC Architecture

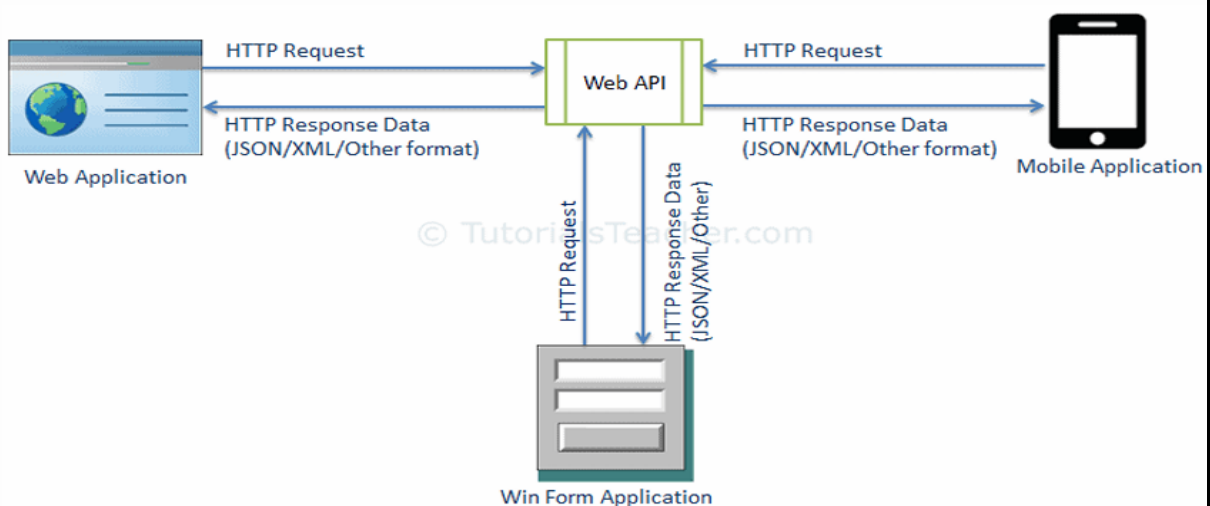
The following figure illustrates the flow of the user's request in ASP.NET MVC.



3. ASP.NET Web API

- **Description:** A framework for building HTTP-based services and RESTful APIs.
- **Features:**
 - Supports multiple formats (JSON, XML)
 - Routing and model binding
 - Statelessness—ideal for modern web and mobile apps

ASP.NET Web API



- **Common Use:** Backend for SPAs, mobile apps, and cross-platform communications

ASP.NET Web API Project

You can create a WebAPI project in two ways.

- WebAPIwith MVC Project
- Stand-alone WebAPI Project

4. ASP.NET Core

- **Description:** The modern evolution of ASP.NET, this is an open-source, cross-platform framework for cloud-based and high-performance web apps.
- **Features:**
 - Runs on Windows, Linux, and macOS
 - Unified programming model (MVC, Web API, and Razor Pages together)
 - Modular and lightweight
 - High performance and scalability

.NET Core Vs ASP.NET Core

.NET Core	ASP.NET Core
Open-source and Cross-platform	Open-source and Cross-platform
.NET Core is a runtime to execute applications build on it.	ASP.NET Core is a web framework to build web apps, IoT apps, and mobile backends on the top of .NET Core or .NET Framework.
Install .NET Core Runtime to run applications and install .NET Core SDK to build applications.	There is no separate runtime and SDK are available for ASP.NET Core. .NET Core runtime and SDK includes ASP.NET Core libraries.
.NET Core 3.1 - latest version	ASP.NET Core 3.1 There is no separate versioning for ASP.NET Core. It is the same as .NET Core versions.

- **Common Use:** Modern web apps, microservices, cloud-native solutions

5. ASP.NET Razor Pages

- **Description:** A page-focused framework within ASP.NET Core, designed for building simple web UI scenarios with a code-behind structure.
- **Features:**
 - Minimal setup, ideal for page-centered apps
 - Encourages clean organization of code and markup

Comparison Table

Technology	Focus/Pattern	Platform	Typical Usage
Web Forms	Event-driven UI	.NET Framework	Business web apps
MVC	MVC pattern	.NET/.NET Core	Modular, testable apps
Web API	RESTful Services	.NET/.NET Core	APIs for web/mobile

Core	Unified/Modern	Cross-platform	Modern web, microservices
Razor Pages	Page-based UI	.NET Core	Simple web UIs

.NET	ASP.NET
.NET is a software development framework aimed to develop Windows, Web and Server based applications.	ASP.NET is a main tool that present in the .NET Framework and aimed at simplifying the creation of dynamic webpages.
Server side and client side application development can be done using .NET framework.	You can only develop server side web applications using ASP.NET as it is integrated with .NET framework.
Mainly used to make business applications on the Windows platform.	It is used to make dynamic web pages and websites using .NET languages.
Its programming can be done using any language with CIL (Common Intermediate Language) compiler.	Its programming can be done using any .NET compliant language.

Mono is an **open-source implementation of Microsoft's .NET Framework**, designed to enable developers to build and run .NET applications across multiple platforms, including Linux, macOS, and Windows. Originally launched by the company Xamarin (formerly Ximian), Mono was created to support cross-platform development and to help bridge the gap between .NET's primarily Windows-focused ecosystem and the broader world of Unix-like operating systems.

Key Points about Mono:

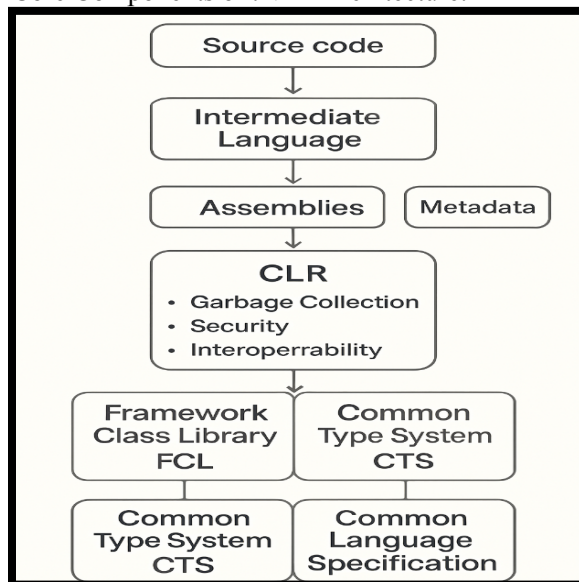
- **Cross-platform:** Allows .NET applications to run on non-Windows systems, such as Linux and macOS.
- **ECMA Compliance:** Implements the ECMA standards for C# and the Common Language Infrastructure (CLI), ensuring compatibility with the official .NET specification.
- **Components:**
 - Mono runtime, which executes applications compiled into Intermediate Language (IL), similar to the CLR (Common Language Runtime) in .NET.
 - Class libraries that correspond to .NET APIs.
- **Use Cases:**
 - Running server applications, web services, and desktop applications developed with .NET on Linux.
 - The engine behind technologies like Unity, which uses Mono for scripting game logic in C#.

- **Legacy and Xamarin:** Although .NET Core and modern .NET provide cross-platform abilities natively, Mono continues to be important for legacy C# apps and mobile development (particularly via Xamarin for Android and iOS).

.NET Architecture and Design Principles

The .NET architecture provides a robust and flexible framework for building a wide range of applications—from desktop and web to mobile, cloud, and IoT. Its design is based on modularity, interoperability, and managed code execution, ensuring security, reliability, and high productivity for developers.

Core Components of .NET Architecture:



1. Common Language Runtime (CLR)

- **Managed Execution:** All .NET applications are executed under the supervision of the CLR, providing services like memory management (garbage collection), type safety, and exception handling.
- **Language Interoperability:** Code compiled for .NET (C#, VB.NET, F#, etc.) is converted into a standard Intermediate Language (IL), which the CLR can execute, allowing seamless integration of different language modules.
- **Security and Safety:** Provides code access security and verification of code safety, with managed execution that helps avoid many common programming errors.

2. Framework Class Library (FCL)

- **Reusable APIs:** A comprehensive collection of reusable classes, interfaces, and value types for handling data, input/output, application configuration, threading, networking, and more.
- **Extensible:** Provides a foundation for higher-level frameworks, such as ASP.NET, Windows Presentation Foundation (WPF), and Windows Communication Foundation (WCF).

3. Assemblies and Metadata

- **Assemblies:** The basic building blocks of .NET applications, compiled as .dll or .exe files, containing IL code and metadata.

- **Metadata:** Describes the types, members, and references in the assembly, supporting features like reflection, late binding, and versioning.

4. Common Type System (CTS) and Common Language Specification (CLS)

- **CTS:** Defines how types are declared and managed in the runtime, ensuring that objects written in different languages can interact.
- **CLS:** A subset of CTS; defines rules for language interoperability so code written in one .NET language can be used by others without conflict.

Key Design Principles

1. Modularity

- Applications can be developed as separate assemblies and modules.
- Encourages code reusability and easier maintenance.

2. Managed Code

- All code runs under CLR control, benefiting from automatic resource management and security.

3. Language Independence

- Supports multiple programming languages; all compile to IL and execute in the CLR environment.

4. Security

- Code Access Security and role-based security models.
- Verifies type safety and enforces permissions at runtime.

5. Performance and Scalability

- Just-In-Time (JIT) compilation converts IL to optimized machine code on demand.
- Garbage collection optimizes memory use, reducing memory leaks.

.NET Application Execution Flow

1. **Source Code Compilation:** Code is compiled into IL (MSIL/CIL), forming assemblies with metadata.
2. **Assembly Loading:** CLR loads the required assemblies.
3. **JIT Compilation:** IL is compiled to native machine code by the CLR's JIT compiler at runtime.
4. **Managed Execution:** The application runs under CLR supervision—enforcing security, type safety, and memory management.

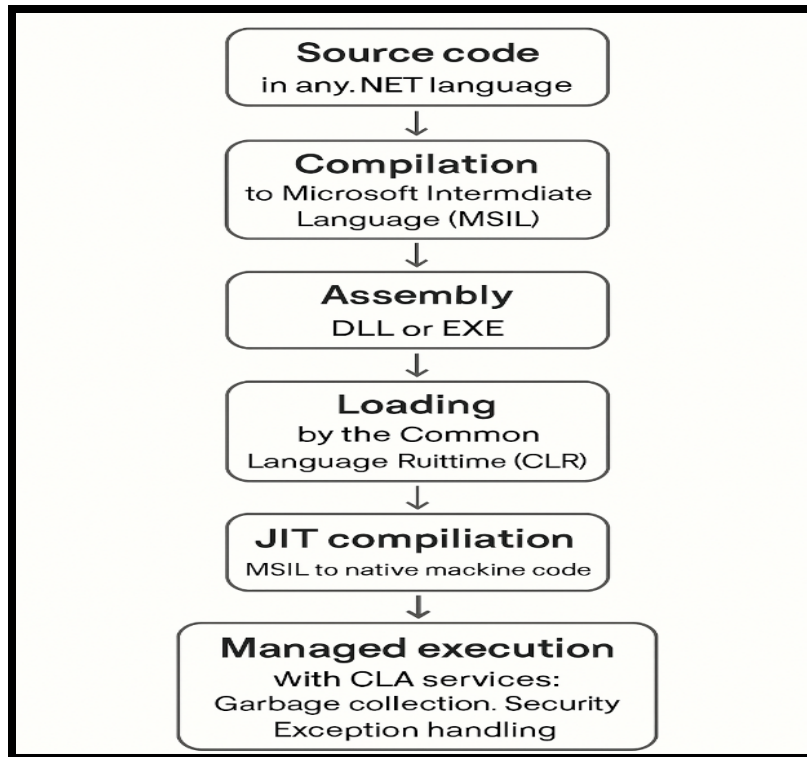
Summary Table

Component	Function
CLR	Managed execution, services (GC, security, etc.)
FCL	Standard reusable libraries/API
Assemblies	Units of deployment, containing IL & metadata
CTS/CLS	Enable multi-language support and interoperability

Compilation and Execution of .NET Applications

Overview

The compilation and execution process in .NET ensures that applications are platform-agnostic, secure, and efficient. This process involves translating source code into an intermediate language and then into machine code at runtime, providing the benefits of managed execution, language interoperability, and optimized performance.



Steps in Compilation and Execution

1. Source Code Compilation

- **Multi-language Support:** .NET supports several programming languages (such as C#, VB.NET, F#).
- **Compiler Translation:** The source code, written in a .NET language, is first compiled by the respective language compiler into Microsoft Intermediate Language (MSIL), also known as Common Intermediate Language (CIL).

2. Creation of Assemblies

- **Assembly Format:** The compiled MSIL code, along with metadata, is packaged into assemblies (files with .dll or .exe extensions).
- **Metadata:** Assemblies contain metadata describing the types, members, and references in the code, which enables features like reflection and versioning.

3. Just-In-Time (JIT) Compilation

- **CLR Involvement:** The Common Language Runtime (CLR) loads the assembly and uses a Just-In-Time (JIT) compiler to convert MSIL/CIL into native machine code suitable for the host operating system and hardware.
- **Execution Process:** The JIT compilation happens at runtime, translating only the methods that are called, which enhances performance.

4. Managed Code Execution

- **Managed Environment:** The code now runs as "managed code," meaning its execution is managed by the CLR.
- **Services Provided by CLR:**
 - Memory management (garbage collection)
 - Exception handling
 - Security enforcement
 - Thread management

- Code verification

Compilation and Execution Workflow

Step	Description
Write Source Code	Code in C#, VB.NET, F#, etc.
Compile to MSIL/CIL	Source is compiled into intermediate language by the language compiler
Create Assembly	MSIL/CIL and metadata packaged into .dll or .exe file
Load Assembly	CLR loads the required assemblies at runtime
JIT Compilation	MSIL/CIL is converted to machine code by the JIT compiler
Execute Managed Code	Application runs, managed by CLR with services like GC and security

Key Technologies

- **Common Language Infrastructure (CLI):** Defines the executable code and runtime environment (including MSIL and CLR).

The **Common Language Infrastructure (CLI)** is an open specification developed by **Microsoft** and standardized by **ECMA** and **ISO**, which describes the **runtime environment** for executing programs written in **multiple high-level languages**.

Key Concepts of CLI:

Term	Description
Purpose	Allows programs written in different languages (like C#, VB.NET, F#) to run on a common runtime environment.
Standard	ECMA-335 and ISO/IEC 23271.
Component of	The .NET Framework and .NET (Core).

Main Components of CLI:

1. **Common Type System (CTS):**
 - Defines how data types are declared and used.
 - Ensures **type safety** across different .NET languages.
2. **Common Language Specification (CLS):**
 - A set of rules that languages must follow to be **CLS-compliant**.
 - Ensures **language interoperability**.
3. **Virtual Execution System (VES):**
 - Executes the compiled code in the **Common Intermediate Language (CIL)**.
 - Manages memory, threads, security, etc.
4. **Metadata:**
 - Stores information about the program's types, members, and references.
 - Enables reflection, type safety, and versioning.
5. **CIL (Common Intermediate Language):**
 - Code compiled from source is converted to CIL.
 - CIL is then **Just-In-Time (JIT) compiled** into native machine code during runtime.

- **MSIL (Microsoft Intermediate Language):** Platform-independent instructions generated from source code.
 - **MSIL**, or **Microsoft Intermediate Language**, is the **low-level, CPU-independent** set of instructions that .NET source code is compiled into before it is executed.
 - After compilation, the source code (e.g., C#, VB.NET) becomes **MSIL**, which is then converted into native machine code by the **Just-In-Time (JIT)** compiler at runtime.

Key Points about MSIL:

Term	Description
Full Form	Microsoft Intermediate Language
Also Called	CIL (Common Intermediate Language) – standardized name
Role	Acts as the bridge between source code and native machine code
Platform	Part of the .NET Framework and .NET Core CLI
Standardized Name	CIL, as per ECMA-335 standard

Features of MSIL:

- **Platform-independent code**
- **Object-oriented instructions**
- **Stack-based execution**
- Supports:
 - Exception handling
 - Arrays
 - Type safety
 - Garbage collection
 - Control flow (if, loops)

How MSIL Works (Execution Flow):

1. **Source Code** (C#, VB.NET, etc.)
↓
2. **Compiler** (e.g., csc.exe)
↓
3. **MSIL Code** (Stored in .exe or .dll with metadata)
↓
4. **JIT Compiler**
↓
5. **Native Machine Code**
↓
6. **Executed by the CPU**

Why Use MSIL?

- Allows **language interoperability** (C#, VB.NET, F#, etc.).
- Enables **cross-platform support** when combined with a compatible runtime.
- Helps optimize runtime performance through **JIT compilation**

- **CLR (Common Language Runtime):** The virtual machine component managing the execution of .NET programs, providing essential services and converting MSIL to machine code at runtime.

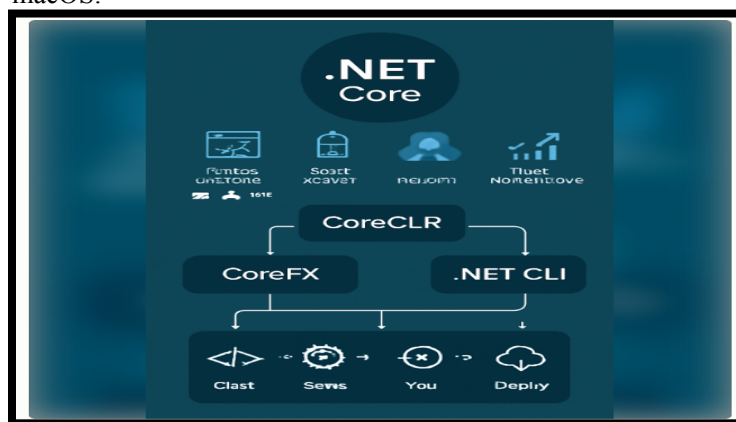
Benefits

- **Portability:** Applications can run on any system with the appropriate CLR implementation.
- **Security:** Managed code execution ensures safe, robust applications.
- **Performance:** JIT compilation translates only necessary code, optimizing speed and resource use.

.NET Core in Detail

Introduction

.NET Core is a modern, open-source, cross-platform framework developed by Microsoft for building a wide range of applications, including web, desktop, cloud, and IoT. It is designed for high performance, modularity, and flexibility, while enabling developers to create scalable solutions that run on Windows, Linux, and macOS.



Key Features

- **Cross-Platform:** Runs on Windows, Linux, and macOS, enabling true platform independence for applications.
- **Open Source:** Actively developed and maintained on GitHub with contributions from Microsoft and the community. Licensed under MIT and Apache licenses.
- **High Performance:** Engineered for speed, scalability, and efficient memory usage, making it suitable for cloud and enterprise-grade applications.
- **Modular Design:** Applications include only the components they need, reducing application footprint and improving maintainability.
- **Unified Development Model:** Supports web (ASP.NET Core), cloud, console, and microservices development from a single codebase.

Architecture

- **CoreCLR:** The runtime responsible for managing application execution, providing services like garbage collection, exception handling, and JIT compilation.
- **CoreFX:** The foundational class libraries, including data types, collections, file systems, and APIs for networking and threading.
- **CLI (Command Line Interface):** Enables developers to create, build, run, test, and deploy .NET Core applications using simple commands, supporting productivity and automation.
- **NuGet Packages:** Functionality is packaged as lightweight NuGet packages, allowing apps to include only what is necessary.

Advantages

- **Portability:** Build once, run anywhere—supports containers and cloud deployments out of the box.
- **Performance:** Outperforms many traditional frameworks in benchmarks, especially for web APIs and microservices.
- **Active Community and Support:** Large, growing ecosystem with continuous releases and community contributions.
- **Modern Approach:** Built-in dependency injection, async/await programming, and easy integration with DevOps pipelines.

Disadvantages

- **Limited Legacy Support:** Does not support some features of the classic .NET Framework (such as Web Forms or some older libraries).
- **Fast Release Cycle:** Requires developers and enterprises to keep up with frequent updates for security and features.
- **Learning Curve:** Some transition required for those used to classic .NET, especially regarding modularity and deployment.

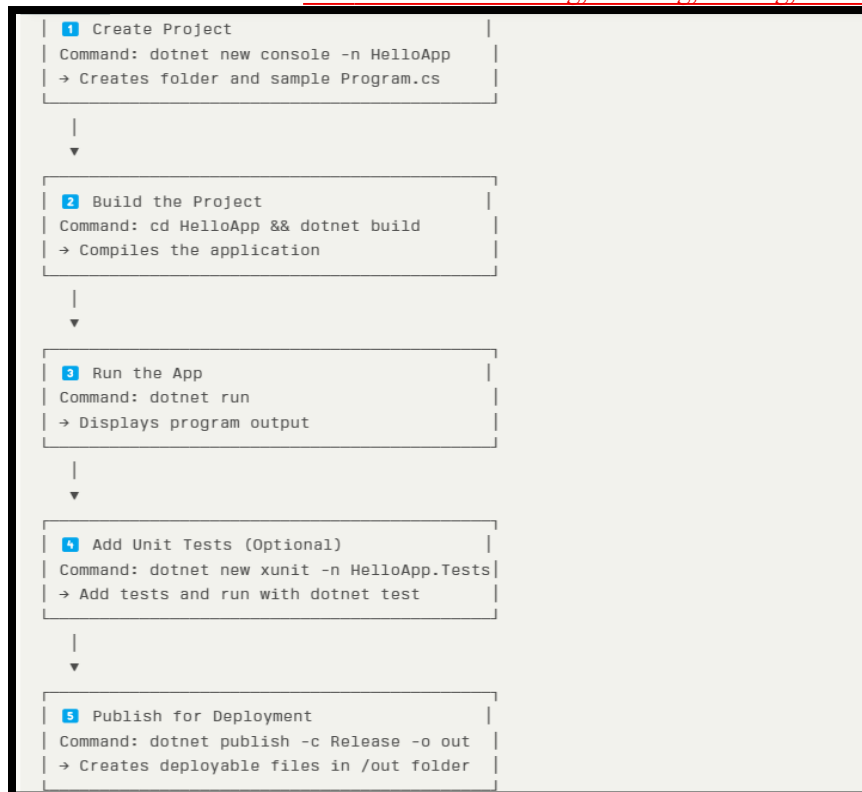
Common Uses

- **Web Applications:** ASP.NET Core for modern, high-performance websites and APIs.
- **Microservices:** Cross-platform deployment and lightweight footprint make it ideal for microservices architectures.
- **Cloud Applications:** Seamless integration with Azure, AWS, and other cloud providers.
- **Command-Line Tools:** Efficient for utilities that run on multiple operating systems.
- **Desktop Apps:** Supports cross-platform desktop apps via .NET MAUI (Multi-platform App UI).

Example CLI Commands

Command	Purpose
dotnet new	Creates a new project
dotnet build	Builds the project
dotnet run	Runs the application
dotnet test	Runs unit tests
dotnet publish	Packages app for deployment

ASP.NET CLI: Building, Running, Testing, and Deploying



Steps:

The .NET Command-Line Interface (CLI) allows you to build, run, test, and deploy .NET Core applications efficiently across platforms. Here are the key operations and commands:

1. Creating a New Project

- **Command:**

`dotnet new <template> -n <ProjectName>`

- **Purpose:** Scaffolds a new project using a specified template (e.g., console, webapi).

2. Building the Application

- **Command:**

`dotnet build`

- **Purpose:** Compiles the application and its dependencies into Intermediate Language (IL) binaries that the .NET runtime can execute.
- **Details:** Automatically checks for code or dependency errors and outputs build artifacts into the /bin directory.

3. Running the Application

- **Command:**

`dotnet run`

- **Purpose:** Builds (if needed) and launches the application directly from source code.
- **Details:** Useful for rapid development and testing as changes are instantly runnable without manual build steps.

4. Testing the Application

- **Command:**

dotnet test

- **Purpose:** Executes all unit tests within the project or solution.
- **Details:** Test results are summarized in the terminal, supporting Test Driven Development and automated checks.

5. Publishing (Deploying) the Application

- **Command:**

dotnet publish -c Release -o <publish-folder>

- **Purpose:** Packages the application and all its dependencies for deployment to any supported platform.
- **Details:** Produces assemblies, configuration files, and optionally platform-specific executables suitable for cloud, server, or container deployment.

Example Program

Located in: HelloApp/Program.cs (auto-generated)

using System;

namespace HelloApp

```
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            Console.WriteLine("Hello, .NET CLI!");  
        }  
    }  
}
```

dotnet new console -n HelloApp

```
Microsoft Windows [Version 10.0.26100.4652]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\rajas\OneDrive - Ridge point\dotnet\UNIT-2-PRACTICAL>dotnet new console -n HelloApp  
  
Welcome to .NET 9.0!  
-----  
SDK Version: 9.0.103  
  
Telemetry  
-----  
The .NET tools collect usage data in order to help us improve your experience. It is collected by Microsoft and shared with the community. You can opt-out of telemetry by setting the DOTNET_CLI_TELEMETRY_OPTOUT environment variable to '1' or 'true' using your favorite shell.  
  
Read more about .NET CLI Tools telemetry: https://aka.ms/dotnet-cli-telemetry  
  
-----  
Installed an ASP.NET Core HTTPS development certificate.  
To trust the certificate, run 'dotnet dev-certs https --trust'  
Learn about HTTPS: https://aka.ms/dotnet-https  
  
-----  
Write your first app: https://aka.ms/dotnet-hello-world  
Find out what's new: https://aka.ms/dotnet-whats-new  
Explore documentation: https://aka.ms/dotnet-docs  
Report issues and find source on GitHub: https://github.com/dotnet/core  
Use 'dotnet --help' to see available commands or visit: https://aka.ms/dotnet-cli  
-----  
The template "Console App" was created successfully.
```

- cd HelloApp

- dotnet build
- dotnet run

```
C:\Users\rajas\OneDrive - Ridge point\dotnet\UNIT-2-PRACTICAL>cd HelloApp

C:\Users\rajas\OneDrive - Ridge point\dotnet\UNIT-2-PRACTICAL\HelloApp>dotnet build
Restore complete (0.4s)
HelloApp succeeded (8.2s) → bin\Debug\net9.0\HelloApp.dll

Build succeeded in 9.4s

C:\Users\rajas\OneDrive - Ridge point\dotnet\UNIT-2-PRACTICAL\HelloApp>dotnet run
Hello, World!

C:\Users\rajas\OneDrive - Ridge point\dotnet\UNIT-2-PRACTICAL\HelloApp>
```

Optional: Add a Test Project

- dotnet new xunit -n HelloApp.Tests
- cd HelloApp.Tests
- dotnet add reference ../HelloApp/HelloApp.csproj
- dotnet test

Publish the App for Deployment

- cd ../HelloApp
- dotnet publish -c Release -o out

Summary Table

Operation	Command Example	Description
Create	dotnet new console -n DemoApp	New project scaffold
Build	dotnet build	Compile source and dependencies
Run	dotnet run	Execute application from source
Test	dotnet test	Run all unit tests
Publish	dotnet publish -c Release -o out	Prepare app for deployment

Key Advantages

- **Cross-platform:** Execute the same commands on Windows, Linux, and macOS.

- **Automation Ready:** Integrate seamlessly into CI/CD pipelines for cloud and enterprise workflows.
- **Rapid Development:** Immediate feedback on changes with dotnet run and dotnet test.

These CLI commands are integral to modern .NET Core and ASP.NET Core development, from initial coding through to cloud deployment.

How to Install a .NET Package (Using NuGet)

You can install a NuGet package in three ways:

1. Using Visual Studio

- Right-click on the project → *Manage NuGet Packages* → Search and Install

2. Using .NET CLI

dotnet add package PackageName

Example:

dotnet add package Newtonsoft.Json

3. Using Package Manager Console

Install-Package PackageName

Example:

Install-Package EntityFramework

How to Check Installed .NET Packages in a Project

Here are different ways to **check which packages are installed** in your .NET project, depending on the development environment and tools you're using:

1. Using Visual Studio (GUI Method)

► Steps:

1. Right-click on the project in **Solution Explorer**.
2. Select "**Manage NuGet Packages...**"
3. Go to the "**Installed**" tab.
4. You'll see all installed packages, including:
 - Package name
 - Version
 - Description

2. Using Package Manager Console (PMC)

► Steps:

1. Go to **Tools > NuGet Package Manager > Package Manager Console**
2. Run the command:

Get-Package

This lists all installed NuGet packages and their versions.

3. Using .NET CLI (for .NET Core/.NET 5/6/7/8 Projects)

► Steps:

In the terminal, run:

dotnet list package

Example Output:

Project 'MyWebApp' has the following package references

[net6.0]:

- > Microsoft.EntityFrameworkCore 6.0.0
- > Newtonsoft.Json 13.0.1

4. By Checking .csproj File

► Steps:

1. Open your project's .csproj file.
2. Look for lines like:

```
<PackageReference Include="Newtonsoft.Json" Version="13.0.1" />
```

```
<PackageReference Include="Microsoft.EntityFrameworkCore" Version="6.0.0" />
```

These lines show which packages are referenced and their versions.

5. By Checking packages.config (for older projects)

► Steps:

1. Open the packages.config file in your project.
2. You'll see entries like:

```
<package id="jQuery" version="3.6.0" />
```

```
<package id="EntityFramework" version="6.4.4" />
```